ClearLedger — План системы учёта

> **Версия:** 2.0 | **Дата:** 2026-02-27
> **Цель:** Буферная зона между реальным миром и 1С:Бухгалтерией — AI-native система приёма, верификации, анализа и подготовки документов
> **Название:** ClearLedger — «Чистая книга». Данные проходят сверку и выходят готовыми к учёту.
> **Первый модуль:** ГСМ (учёт топлива на сети АЗС)

Маркетинговая концепция: 3 кейса ценности

ClearLedger — **буферная зона между реальным миром и 1С:Бухгалтерией**. Всё, что попадает в бухгалтерию клиента, уже проверено, сверено и подтверждено. Система решает три последовательных задачи, каждая из которых — самостоятельный продаваемый кейс:

Кейс 1: Приём и верификация первичных документов

Боль: Документы приходят из 5+ источников в разных форматах. Ошибки находят при закрытии периода, когда поздно. Ручная проверка каждого документа — часы работы бухгалтера.

Решение:

- Автоматический приём из любых каналов (email, сканы, 1С XML, Excel, мессенджеры, API)
- Извлечение метаданных, текста, построение графа связей между документами
- **Сверка с эталоном** — данные клиента из 1С:Бухгалтерии (контрагенты, договоры, номенклатура, ИНН/КПП) используются как образец для верификации
- **Внешние сервисы** (Слой 2+): проверка контрагента через ФНС, Rusprofile, судебные базы — благонадёжность, банкротство, массовый адрес, дисквалифицированные лица
- Результат: документ либо подтверждён, либо помечен с конкретными ошибками и рекомендациями

USP: «Не мы говорим что документ правильный — ваша же 1С это подтверждает»

Кейс 2: AI-анализ незакрытого периода

Боль: Бухгалтер видит ошибки только при закрытии квартала. Акты сверки с контрагентами делают раз в квартал вручную. Расхождения копятся.

Решение:

- Нейросеть обучена на учётной политике и правилах конкретного клиента (Skills)
- Непрерывный анализ документов по незакрытому периоду — ДО загрузки в 1С
- Автоматические сверки со ВСЕМИ контрагентами: акты сверки, расхождения сумм, дат, номенклатуры
- Нормализация: одни и те же контрагенты, номенклатура, ставки — единообразно
- Все данные верифицированы, проверены и подтверждены перед передачей в 1С

USP: «Проблема найдена ДО того, как она попала в 1С»

Кейс 3: Моделирование закрытия периода и рекомендации

Боль: Закрытие квартала — стресс, ошибки, штрафы. Бухгалтер не видит общей картины до последнего момента. Нет инструмента «а что если».

Решение:

- «Песочница»: клиент видит, как будет выглядеть закрытие периода с текущими данными
- С учётом проводок, учётной политики, налоговых правил — в трёх контурах (управленческий, бухгалтерский, налоговый)
- Рекомендации: «не хватает акта от ООО "Альфа"», «расхождение НДС на 45 000 ₽», «ретро-бонус не учтён»
- Анализ данных против бизнес-правил и налогового законодательства → предложения по оптимизации учёта, отношений с контрагентами, взаимодействия с государством

USP: «Закройте квартал мысленно, прежде чем нажать кнопку в 1С»

Связь кейсов с архитектурой

```
Кейс 1 → Слой 1 (RAW) + Слой 2 (нормализация)     ← РЕАЛИЗУЕТСЯ СЕЙЧАС
Кейс 2 → Слой 2 (сверки) + AI Skills                ← СЛЕДУЮЩИЙ ЭТАП
Кейс 3 → Слой 3 (Clean Docs) + Accounting Kernel    ← ГОРИЗОНТ
```

Отстройка от конкурентов

Конкурент	Что делает	ClearLedger отличается
1С:Бухгалтерия	Учёт факта	Мы — ДО факта: проверка, верификация, моделирование
Контур.Диадокс	ЭДО — доставка документов	Мы — содержание: проверяем ЧТО в документе, а не КАК доставить
СБИС / Rusprofile	Проверка контрагентов (статика)	Мы — динамическая сверка в контексте ваших операций с контрагентом
Entera.pro	OCR → загрузка в 1С	Мы — OCR + верификация + сверки + анализ + моделирование
BlackLine / Duco	Reconciliation (enterprise)	Мы — для 1С экосистемы, в 50× дешевле, с AI-навыками

> **Маркетинговая политика, защита IP и монетизация** — детальный стратегический документ: MARKETING_AND_IP_STRATEGY.md. Включает: архитектуру защиты «умная коробка + облачный мозг», 7 уровней защиты IP, тарифную сетку, каналы продаж, юридическую защиту (патент, TM, SaaS-лицензия, ноу-хау), риски и митигацию.

Философия

ClearLedger = мозг учёта, 1С = бухгалтерия и отчётность.

ClearLedger принимает на вход любые разрозненные данные (API, CSV, PDF, Excel, email) и с помощью **AI-навыков** — а не зашитых в код правил — приводит их к нормальному виду, сверяет, находит оптимальные решения для учёта и формирует готовые документы для 1С.

Ключевое отличие от классических систем: логика обработки данных живёт не в коде, а в **навыках** (skills) — описаниях на естественном языке, которые AI использует для понимания и нормализации данных. Новый тип данных = новый навык, а не новый релиз. При этом детерминированные операции (парсинг, математика, валидация) выполняет **код на Python** — мгновенно и бесплатно. AI подключается там, где нужно суждение, контекст или интерпретация.

ClearLedger даёт бизнесу **три уровня учёта** из одних и тех же первичных данных:

- **Управленческий** — реальная картина для принятия решений (маржа, потери, KPI)
- **Бухгалтерский** — по правилам ПБУ/ФСБУ для официальной отчётности

- **Налоговый** — подсветка вариантов отражения, рисков и рекомендаций для налогового специалиста

AI анализирует первичные документы и договоры, **подсвечивает варианты трактовки и риски**. Финальное решение по учёту — за бухгалтером/налоговым специалистом, с обязательной фиксацией основания. В 1С уходят выверенные документы — 1С сама делает проводки. Ручные правки в 1С запрещены: не так — исправляем в ClearLedger, выгружаем заново.

Почему не Excel

Excel — не альтернатива, а источник проблем:

- **Нет истории:** кто-то перезаписал ячейку — данные потеряны навсегда. ClearLedger хранит каждое изменение: кто, когда, зачем, старое и новое значение.
- **Нет валидации:** формула сломалась — никто не заметил до конца месяца. ClearLedger проверяет данные в момент загрузки.
- **Нет автоматизации:** менеджер вручную копирует из одного окна в другое. ClearLedger забирает данные по API сам.
- **Нет AI:** Excel не умеет анализировать паттерны и предлагать корректировки. AI-агент стоит ~$2.5/месяц на Haiku и заменяет часы ручной сверки.
- **Зависимость от человека:** менеджер заболел — никто не знает, как работает его файл. ClearLedger работает без конкретного человека.
- **Скрытые потери:** ошибки, которые находят через месяц при сверке → корректировки задним числом → штрафы. Дороже любой системы.

Почему AI нужен с первого дня

Расхождения в данных не зависят от количества АЗС. Одна станция за одну смену: касса может не сойтись с чеками, банковский реестр задержался, ТТН не совпала с фактом приёмки, остаток в резервуаре уплыл. Это природа бизнеса с наличными, физическим товаром и несколькими системами оплаты.

AI в 2026 — не роскошь, а базовый инструмент. Стоимость AI-вызова на порядки ниже ручной работы (при гибридном подходе, где код обрабатывает 95% операций, а AI подключается только к расхождениям — подробный расчёт в секции 16). Точная экономика зависит от размера контекста, количества ретраев и edge-cases, но порядок разницы — **десятки-сотни рублей AI vs десятки тысяч рублей ручного труда** в месяц.

Когда ClearLedger, а когда 1С:Торговля / Битрикс24

ClearLedger — **не замена** 1С и не замена CRM. Это **слой между хаосом данных и чистой бухгалтерией**. Вопрос не "ClearLedger или 1С", а "нужен ли ClearLedger перед

1С".

Ключевой критерий: откуда приходят данные?

```
Данные из одного источника       → 1С:Торговля справится сама
Данные из 2–3 связанных систем   → 1С с доработкой, возможно
Данные из 4+ разнородных систем  → ClearLedger + 1С
```

Когда хватает 1С:Торговля

Ситуация	Почему 1С достаточно
Магазин: 1 касса, 1 банк, 1 поставщик	Данные вводятся в 1С напрямую, сверять нечего
Оптовая торговля со складом	1С:Торговля = склад + продажи + закупки в одной системе
Типовой бизнес с типовыми операциями	1С покрывает 95% сценариев «из коробки»
Всё происходит внутри одной системы	Нет разрозненных источников = нет проблемы сверки

1С:Торговля — полноценная ERP: склад, закупки, продажи, ценообразование, CRM, HR. Если бизнес укладывается в её модель — ClearLedger избыточен. Лицензия 30-100К₽ + внедрение 200-500К₽, но получаешь готовую систему с экосистемой из тысяч специалистов.

Когда хватает Битрикс24

Ситуация	Почему Битрикс достаточно
Нужен CRM + воронка продаж + задачи	Битрикс — про управление клиентами, не про учёт
Команда работает в чатах/задачах	Коммуникации + проекты + базовый документооборот
Простое выставление счетов	Битрикс → счёт → оплата → 1С (стандартная интеграция)

Битрикс24 — это CRM и коммуникации. Он не решает задачу учёта. Если основная боль — продажи и командная работа, Битрикс закрывает потребность. Связка Битрикс→1С работает для простых сценариев.

Когда нужен ClearLedger

Ситуация	Почему 1С/Битрикс не справляются
Сеть АЗС: STS + банки + топл. карты + MSTO + ТТН + резервуары	6-7 разных источников. 1С не подключается к STS API. Нет сверки топливных карт с показаниями ТРК
Сеть магазинов с разными ПО на кассах	Кассы разных производителей, разные форматы. 1С ожидает единый формат
Франшиза: данные от франчайзи в разных форматах	Каждый франчайзи присылает отчёты по-своему. 1С не парсит произвольные Excel/PDF
Бизнес с физическим товаром + потерями	Нормативные потери, пересортица, усушка — 1С не считает потери по нормативам Минэнерго
Много способов оплаты: наличные, карты, СБП, топл. карты, бонусы	Каждый канал — отдельный реестр, нужна перекрёстная сверка
Сложные договоры: ретро-бонусы, пороги, условные скидки	1С не анализирует условия договоров для оптимизации учёта

Паттерн: ClearLedger нужен когда есть **проблема сверки** — данные приходят из разных мест в разных форматах, и кто-то (раньше менеджер, теперь AI) должен их собрать, сверить и подготовить для бухгалтерии.

Честное сравнение

```
                        1С:Торговля    Битрикс24    ClearLedger
──────────────────────────────────────────────────────────────────
Тип системы             ERP            CRM          Reconciliation layer
Учёт товаров            ✓ полный       ✗            ✗ (не задача)
CRM / воронка           ✓ базовый      ✓ полный     ✗ (не задача)
Бухгалтерский учёт      ✓ (связка)     ✗            ✗ (это делает 1С)
Налоговая отчётность    ✓ (связка)     ✗            ✗ (это делает 1С)

Подключение к произ-
вольным API             ✗              ✗            ✓
Парсинг произвольных
форматов (CSV/PDF/XML)  ✗              ✗            ✓ (AI + код)
Перекрёстная сверка
5+ источников           ✗              ✗            ✓
AI-анализ расхождений   ✗              ✗            ✓
AI-анализ договоров     ✗              ✗            ✓
Три контура учёта
```

```
из одних данных          ✗                ✗              ✓
Онбординг за 2-4 недели  ✗ (месяцы)       ✓ (SaaS)       ✓ (AI Discovery)
Адаптация без кода       ✗ (1С-разраб.)   ✗ (разраб.)    ✓ (навыки)

Стоимость/мес (10 АЗС)   ~50К₽ подд.      ~12К₽          ~3К₽ + 1С
Внедрение                200-500К₽        50-100К₽       MVP 4-6 нед.
Экосистема               огромная         большая        нет (пока)
```

Вывод: ClearLedger — это прослойка, а не замена

```
Простой бизнес:
  Данные → 1С:Торговля → Бухгалтерия → Отчётность
  (ClearLedger не нужен)

Сложный бизнес:
  Данные из 5+ систем → ClearLedger → Выверенные документы → 1С → Отчётность
                              ↑
                      AI сверяет, оптимизирует,
                      готовит документы
```

ClearLedger не заменяет 1С — он **решает проблему, которую 1С не решает**: собрать разрозненные данные, сверить их между собой, найти расхождения, предложить оптимальный учёт и сформировать чистовые документы. 1С остаётся на своём месте — бухгалтерия и отчётность.

Если у бизнеса нет проблемы сверки — ClearLedger не нужен. Если есть — 1С и Битрикс её не решат.

Главный конкурент в России — не Duco, а «МарьИванна»

Самый опасный конкурент ClearLedger — не BlackLine за $350K/год и не Duco за $52K. Это **менеджер, который «и так всё знает»** и Excel-файл, который «работает уже 5 лет».

Но есть проблема глубже: ClearLedger делает данные **прозрачными**. Каждая операция сверена, каждое расхождение зафиксировано, каждая корректировка видна. Если кто-то в цепочке (управляющий АЗС, менеджер по закупкам, кладовщик) **заинтересован в непрозрачности** — он будет саботировать внедрение.

Следствие для продаж:

- Продавать **собственнику**, а не операционному директору
- ClearLedger — инструмент **контроля над менеджментом**, а не инструмент для менеджмента

- Первый аргумент: «Вы знаете реальные потери на своих АЗС? А ваш управляющий знает, что вы не знаете?»
- Прозрачность — это фича для собственника и проблема для того, кто «химичит»

Стратегия внедрения: параллельная работа (Фаза 4 онбординга) снимает саботаж. Систему не надо «внедрять» — она просто работает рядом и показывает результаты. Когда собственник видит расхождения, которые раньше «не замечали», решение о переключении принимает он, а не менеджер.

Конкурентный ландшафт (февраль 2026)

Рынок reconciliation-софта: **$3.5 млрд** (2024) → **$8.9 млрд** (2033). 95% CFO уже инвестируют в AI для сверки. Но ни один продукт не закрывает нашу нишу целиком.

Глобальные enterprise-платформы

Продукт	Что делает	AI	Цена	Слабость vs ClearLedger
BlackLine	Financial close, reconciliation, compliance	ML	$50K-350K/год	Дорого, месяцы внедрения, жёсткие rules, нет CSV/PDF ingestion
Trintech	Financial close для mid-market	ML	Кастом	Фокус на compliance, не cross-source reconciliation
ReconArt	Web-based reconciliation	Нет	$300/мес/юзер	Классический rules-based, нет AI, нет неструктурированных данных
HighRadius	AR automation, treasury, reconciliation	ML	Enterprise	Enterprise-only, нет гетерогенных бизнес-источников

Общая слабость: Все ориентированы на financial close (закрытие периода внутри ERP). Не решают задачу «собрать данные из 5 разных внешних систем и сверить». Ни один не работает с 1С.

AI-native стартапы (2023-2026) — прямые конкуренты по подходу

Продукт	Раунд	Что делает	Чем близок	Чем отличается
Duco	—	No-code reconciliation, 170+ форматов	AI Rule Builder на plain English	$52K+/год, capital markets, нет 1С

Продукт	Раунд	Что делает	Чем близок	Чем отличается
Simetrik	$85M	No-code reconciliation, 1B записей/день	Agentic AI, Excel-like UI	Fintechs/payments, нет бизнес-источников
Digits	$100M+	Autonomous General Ledger (замена QuickBooks)	AI bookkeeping agents	Не reconciliation layer, US only
Basis	$100M	AI agents для accounting firms (OpenAI o3/GPT-5)	Мощный LLM stack	Для бухг. фирм, не для бизнеса, US only
DualEntry	$90M	AI-native ERP (замена NetSuite)	Полный AI-ERP за 24 часа	Конкурирует с ERP, не reconciliation layer
Docyt	$12M	AI bookkeeping, 30+ POS интеграций	Multi-POS, вертикали	US/QuickBooks, не cross-source reconciliation
Ledge.co	—	AI agents для payment reconciliation	AI edge-case handling	Только payments
Leapfin	—	Natural language prompts для workflows	Близкая концепция NL-правил	Revenue data только, нет 1C
Numeric	$51M	AI matching (90%+ vs 30% индустрия)	AI-powered matching	Financial close, не бизнес-источники

Ключевое наблюдение: В AI-accounting вложено >$500M за 2025-2026. Но все стартапы нацелены на **US market** (QuickBooks/NetSuite/SAP) и на **payments/financial close**. Никто не решает задачу **гетерогенных бизнес-источников** (API станций + CSV банков + PDF ТТН + топливные карты) для **1С-экосистемы**.

Ближайший конкурент по идеологии: Duco

Duco — единственный продукт с **AI Rule Builder на natural language** + **multi-format ingestion** (170+ форматов). Заявляет 50% ускорение создания правил, setup за 3 часа вместо 74 дней.

Почему Duco — не конкурент для нашего рынка:

- Цена: от $52K/год (мин.) до $281K/год — это enterprise capital markets
- Фокус: банки, хедж-фонды, платёжные системы — не retail/fuel/SMB
- Нет 1С — даже теоретически

- AI rules ≠ AI skills: у Duco AI создаёт matching rules, у ClearLedger AI **понимает бизнес-контекст** (договоры, нормативы, три контура учёта)

Российский рынок — пусто

Продукт	Что делает	Слабость
1С:Сверка 2.0	Сверка документов контрагентов внутри 1С	Только внутренняя сверка, нет внешних источников, нет AI
Entera.pro	OCR первичных документов → загрузка в 1С	Только ingestion (распознавание), нет reconciliation
СБИС / Диадок	ЭДО (передача документов)	Транспорт, не сверка
1С:Упр. автотрансп. (Rarus)	Загрузка топливных карт + ГЛОНАСС	Узкий транспортный фокус, нет AI, нет cross-source

В России нет ни одного AI-native reconciliation layer. Entera.pro ближе всех (OCR + 1С), но решает только задачу распознавания документов, не сверки данных из нескольких источников.

Топливная вертикаль — legacy

Продукт	Рынок	Слабость
PDI Technologies	US fuel wholesale, 250+ маркетёров	Monolithic ERP, нет AI reconciliation
AIMS	US petroleum accounting, SIR	Legacy, нет AI, нет multi-source
DigitalPetro	IoT сверка объёмов топлива	Только физическая сверка, нет финансов

Все — US market, legacy architecture, без AI.

Где ClearLedger — единственный

```
                          AI-native  Гетерогенные  1С       Доступная
                          reconcil.  бизнес-источ. экосист.  цена (<$1K/мес)
 BlackLine                  –           –           –          –
 Duco                       ✓           ✓           –          –
 Simetrik                   ✓           –           –          –
 Digits/Basis/DualEntry     ✓           –           –          ✓
```

```
Entera.pro                  –           –           ✓           ✓
1C:Сверка                   –           –           ✓           ✓
ClearLedger                 ✓           ✓           ✓           ✓
```

ClearLedger занимает **незанятое пересечение** четырёх осей: AI-native + гетерогенные источники + 1С + доступная цена. Ни один из 30+ исследованных продуктов не закрывает все четыре.

Риски от конкурентов

Угроза	Вероятность	Горизонт	Наш ответ
1С встроит AI reconciliation нативно	Средняя	2-3 года	У нас фора: 1С исторически медленно внедряет AI. Мы будем первыми с рабочим продуктом
Duco/Simetrik выйдут на российский рынок	Низкая	3-5 лет	Санкции, нет 1С интеграции, ценовой сегмент не совпадает
Entera.pro добавит reconciliation	Средняя	1-2 года	У них OCR-ядро, не reconciliation архитектура. Другая ДНК
Новый российский стартап скопирует подход	Средняя	1-2 года	Библиотека навыков = конкурентный барьер. Каждый клиент делает нас умнее

---

1. Архитектура: три слоя + 1С

```
СЛОЙ 1          СЛОЙ 2              СЛОЙ 3           1С
RAW DATA    →   DOCUMENTS       →   CLEAN DOCS   →   БУХГАЛТЕРИЯ
(сырое)         (нормализация,       (готовые для      (проводки,
                 сверка,              выгрузки)         отчётность)
                 корректировка)
```

Слой 1: RAW (сырые данные)

Что: Всё, что приходит извне — как есть, без изменений.

Решение: Таблица `raw_data`, append-only, JSONB payload.

Почему так:

- Иммутабельность — всегда можно вернуться к оригиналу и перепарсить

- AI учится на истории сырых данных — видит, как менялись форматы
- При ошибке парсинга — данные не потеряны, можно исправить парсер и прогнать заново

```
Источники:
  STS API (JSON)      → транзакции, смены, резервуары, ТТН
  Банковский реестр    → CSV эквайринга
  TradeCorp API        → корпоративные карты
  MSTO API             → онлайн-заказы
  PDF / email          → ТТН от поставщиков (OCR)
  Excel                → прайсы, ручные данные
```

Слой 2: DOCUMENTS (нормализация + сверка)

Что: Единая структура для всех типов документов. Здесь живёт вся логика: парсинг, сверка, корректировка, верификация.

Решение: Таблица `documents` с гибридной моделью (колонки + JSONB metadata).

Почему гибрид, а не чистый SQL или чистый NoSQL:

- Колонки (doc_type, doc_date, station_id, status) — для фильтрации, JOIN, индексов. Без этого запросы по JSONB медленные.
- JSONB metadata — для специфики каждого типа документа. Сменный отчёт и ТТН имеют разную структуру, но живут в одной таблице. Новый тип документа — не нужна миграция БД.
- AI читает JSONB напрямую — самодокументируемая структура, не нужно описывать схему из 50 таблиц.

Статусная модель:

```
draft → validated → verified
         ↑ корректировка (возврат на draft)
```

- `draft` — документ создан из RAW, ждёт сверки
- `validated` — автосверка пройдена (может быть с предупреждениями)
- `verified` — менеджер подтвердил, документ готов к слою 3

Слой 3: CLEAN DOCS (готовые для 1С)

Что: Выверенные документы в формате, понятном 1С. Без проводок — 1С делает их сама.

Решение: Таблица `accounting_documents` + XML-генератор.

Почему документы без проводок:

- 1С имеет учётную политику и регистр "СчетаУчётаНоменклатуры" — она сама знает, на какие счета разносить
- Передача готовых проводок (Дт/Кт) ломает целостность учёта в 1С: если поменяется учётная политика, наши проводки станут неправильными
- Мы передаём **что произошло** (продали 1240л АИ-92 за наличные), а 1С решает **как отразить** (Дт 50.01 Кт 90.01.1)
- Единственное исключение: документ "Операция" для ручных бухгалтерских корректировок

Что ClearLedger знает о проводках и зачем (v2.0+):

- Accounting Kernel формирует **ожидаемые проводки** для управленческого учёта и контроля
- После проведения в 1С — сравниваем ожидаемое с фактическим (feedback)
- Если расхождение — сигнал: либо учётная политика 1С настроена неправильно, либо маппинг ClearLedger устарел

> **Граница scope:** Полноценный Accounting Kernel (план счетов, маппинг проводок, мультиполитики) — это по сложности отдельный продукт. В MVP/v1 Слой 3 = XML-генератор + feedback из 1С. Ожидаемые проводки — горизонт v2.0+, когда будет реальный feedback от первых клиентов. Кейс 1 + Кейс 2 уже достаточны для монетизации. Подробнее о стратегии монетизации → MARKETING_AND_IP_STRATEGY.md.

Статусная модель:

```
ready → exported → posted
                      ↑ feedback из 1С
         ↓ (ошибка)
       error → исправление в слое 2 → новый документ → ready
```

1С: только бухгалтерия и отчётность

Что делает 1С:

- Принимает XML-документы (продажи, поступления, списания, инкассация)
- Создаёт документы по своим правилам
- Формирует проводки по своей учётной политике

- Проводит документы
- Формирует отчётность (баланс, ОПиУ, декларации, книги покупок/продаж)
- Отправляет feedback (успех/ошибка/номер документа)

Что 1С НЕ делает:

- Не сверяет данные (это ClearLedger)
- Не корректирует суммы (это ClearLedger)
- Не принимает ручных правок от бухгалтера (запрещено)

---

2. Навыки (Skills) вместо правил в коде

Принцип

Классический подход: бизнес-логика зашита в код. Новый тип данных → новый парсер → новый релиз → тестирование → деплой. Медленно, дорого, требует разработчика.

Подход ClearLedger: Бизнес-логика живёт в **навыках** — описаниях на естественном языке, которые AI использует для обработки данных. Новый тип данных → новый навык (текстовый файл) → AI сразу умеет работать.

Что такое навык

Навык — это инструкция для AI, описывающая:

- Какие данные на входе (формат, источник, структура)
- Как их интерпретировать (бизнес-смысл полей)
- Как нормализовать (в какую структуру привести)
- Какие сверки выполнить (с чем сравнить, какие допуски)
- Как отразить в каждом контуре учёта (управленческий, бухгалтерский, налоговый)

```
skills/
├── sources/                    # Навыки по источникам данных
│   ├── sts_shift_report.md     # Как парсить сменный отчёт STS
│   ├── sts_transactions.md     # Как парсить транзакции STS
│   ├── bank_acquiring.md       # Как парсить банковский реестр эквайринга
│   ├── ttn_pdf.md              # Как извлечь данные из PDF ТТН
│   └── tradecorp_cards.md      # Как парсить операции по топл. картам
│
├── reconciliation/             # Навыки сверки
│   ├── cash_vs_receipts.md     # Сверка кассы с чеками
```

```
│   ├── bank_vs_acquiring.md    # Сверка банка с эквайрингом
│   ├── tank_balances.md        # Сверка остатков резервуаров
│   └── ttn_vs_fact.md          # Сверка ТТН с фактом приёмки
│
├── accounting/                 # Навыки учёта
│   ├── fuel_retail_sale.md     # Как отражать розничную продажу ГСМ
│   ├── fuel_receipt.md         # Как отражать поступление ГСМ
│   ├── fuel_loss.md            # Как отражать потери (норма vs сверхнорма)
│   ├── cash_collection.md      # Как отражать инкассацию
│   └── corporate_sale.md       # Как отражать корп. продажи по картам
│
└── contracts/                  # Навыки работы с договорами
    ├── supply_contract.md      # Как учитывать условия договора поставки
    ├── franchise_contract.md   # Как учитывать франшизу
    └── lease_contract.md       # Как учитывать аренду АЗС
```

Пример навыка

```
# Навык: Розничная продажа ГСМ

## Входные данные
Сменный отчёт из STS API (doc_type: shift_report) в статусе verified.

## Управленческий учёт
- Выручка = сумма реализации по каждому виду топлива
- Себестоимость = средневзвешенная закупочная × литры
- Маржа = выручка − себестоимость
- Разбивка по способам оплаты: наличные, карты, топл. карты, онлайн

## Бухгалтерский учёт (ПБУ/ФСБУ)
Документ: Отчёт о розничных продажах
- Каждый вид топлива — отдельная строка
- НДС рассчитывается по ставке 20/120 (включён в розничную цену)
- Способ оплаты определяет тип документа оплаты:
  - Наличные → отражается в кассовом отчёте
  - Банковские карты → эквайринг (57.03 до зачисления на р/с)
  - Топливные карты → дебиторка по контрагенту
  - Онлайн (СБП) → эквайринг

## Налоговый учёт
- Выручка признаётся в момент отпуска товара (дата смены)
- НДС исходящий — в книгу продаж за дату смены
- Если есть договор с контрагентом на топл. карты:
  проверить условия — может быть отсрочка признания выручки
```

```
## Рекомендации AI (требуют утверждения специалистом)
- Потери в пределах норм (Минэнерго №527):
  → Вариант: списать на расходы (НК РФ ст. 254 п.7). Основание: [ссылка на норматив]
- Потери сверх нормы:
  → Подсветка: НЕ уменьшают налоговую базу. Рекомендация: отразить отдельно
- Акт инвентаризации за период:
  → Подсветка вариантов перераспределения. **Решение — за налоговым специалистом**
- Каждая рекомендация фиксирует: версию навыка, основание, ссылку на НПА
```

Почему навыки, а не код

Правила в коде	Навыки (Skills)
Новый тип данных = разработчик + релиз	Новый тип данных = текстовый файл
Изменение логики = код + тесты + деплой	Изменение логики = редактирование навыка
Понимает только разработчик	Понимает бухгалтер, менеджер, руководитель
Жёсткая структура: if/else	Гибкая: AI интерпретирует контекст
Не учитывает договоры	AI читает условия договора и применяет
Один способ отражения	AI подсвечивает варианты для каждого контура

Ограничения навыков (что skills НЕ могут)

Навыки — мощный инструмент, но с жёсткими границами:

1. **Skills не меняют детерминированные вычисления.** Расчёт НДС, суммирование, проценты — это код. Навык может задать допуск сверки (0.3% или 0.5%), но не формулу расчёта НДС.
2. **Skills не обходят Pydantic-валидацию.** Если навык предлагает структуру, которая не проходит схему документа — отказ. Навык конфигурирует допуски и решения, а не формат данных.
3. **Каждое решение AI ссылается на основание:**
   - Версия навыка (skill_version: "fuel_retail_sale@v2.3")
   - Ссылка на НПА / договор / норматив
   - Конкретные данные, на основе которых принято решение

4. **Версия навыка записывается в документ.** Если навык обновился — можно найти все документы, обработанные старой версией, и перепроверить.
5. **Skills не заменяют подпись специалиста.** Налоговые и бухгалтерские рекомендации AI — это подсветка вариантов. Финальное решение утверждает человек с соответствующей ответственностью.

```
-- В каждой версии документа фиксируется:
changes JSONB = {
  "field": "tax_treatment",
  "old": null,
  "new": "loss_within_norm",
  "reason": "Потери 0.3% < норма 0.5% (Минэнерго №527)",
  "skill": "accounting/fuel_loss@v1.2",
  "approved_by": null,          -- ждёт утверждения
  "npa_ref": "НК РФ ст.254 п.7"
}
```

Регрессионные тесты навыков

При изменении навыка — автоматический прогон на **golden cases**:

```
skills/
├── accounting/
│   ├── fuel_retail_sale.md
│   └── fuel_retail_sale.tests/
│       ├── golden_case_001.json   # Типовая смена, всё сходится
│       ├── golden_case_002.json   # Расхождение кассы, AI предлагает
корректировку
│       ├── golden_case_003.json   # Потери сверх нормы
│       └── expected_results.json  # Ожидаемые решения AI для каждого кейса
```

Метрики качества (считаются автоматически):

Метрика	Что измеряет	Целевой порог
Precision (матчинг)	% правильных сопоставлений из предложенных	>95%
Recall (матчинг)	% найденных сопоставлений из существующих	>90%
Ручные правки / 100 смен	Сколько раз менеджер поправил AI	<5

Метрика	Что измеряет	Целевой порог
AI-rejection rate	% отклонённых рекомендаций AI	<10%
Drift detection	Ухудшение метрик после обновления навыка	Алерт при Δ>5%

Прогон golden cases — обязательный шаг перед активацией обновлённого навыка. Если метрики упали — навык не активируется, администратор получает алерт.

Workflow изменений навыков (role-based)

Чтобы изменения навыков были управляемыми, вводим процесс как в code review:

1. Автор (аналитик/админ) делает черновик `skill@draft`.
2. Автотесты: golden cases + метрики качества + проверка ссылок на основания.
3. Ревью: бухгалтер/налоговый специалист подтверждает корректность трактовки.
4. Approve: руководитель контура (учёт/налоги) переводит в `ready_for_canary`.
5. Canary: ограниченный запуск на части документов.
6. Promote или rollback: массовая активация только после стабильных метрик.

Роли:

Роль	Что делает
Автор навыка	Редактирует текст навыка, указывает основание
Ревьюер учёта	Проверяет бухгалтерскую корректность
Ревьюер налогов	Проверяет налоговые трактовки
Администратор	Запускает canary/promote/rollback

Canary / A-B перед массовой активацией

Обновлённый навык активируется по схеме 10% → 50% → 100% документов на типе `doc_type`.

Критерии остановки canary:

- рост AI-rejection rate > 3 п.п. к baseline;
- рост ручных правок > 2 на 100 документов;
- новая критическая ошибка в налоговом/бухгалтерском контуре.

Если условие срабатывает — автопереход в rollback на предыдущую версию навыка.

Как это работает

```
Входные данные (JSON/CSV/PDF)
        │
        ▼
AI загружает навык источника (sources/sts_shift_report.md)
        │ "Как парсить эти данные"
        ▼
Нормализованный документ (JSONB в documents)
        │
        ▼
AI загружает навыки сверки (reconciliation/*.md)
        │ "С чем и как сверять"
        ▼
Результат сверки + предложения по корректировке
        │
        ▼
AI загружает навыки учёта (accounting/fuel_retail_sale.md)
        │ "Как отразить в каждом контуре"
        │
        ├──► Управленческий: маржа, KPI, аналитика
        ├──► Бухгалтерский: документ для 1С
        └──► Налоговый: варианты + рекомендации
                │
                ▼
        Если есть договор → AI загружает навык договора
                │ "Какие условия влияют на учёт"
                ▼
        Финальный документ для 1С (XML)
```

Навыки + договоры = оптимизация

AI читает не только первичные данные, но и **условия договоров**:

```
Пример: Договор поставки ГСМ с ООО "НефтеСнаб"

Условия:
- Переход права собственности: в момент слива в резервуар
- Допустимая недостача при транспортировке: 0.5% от массы
- Ретро-бонус: 2% при объёме закупок >100 тонн/месяц
- Отсрочка оплаты: 14 календарных дней

AI применяет:
- Управленческий: оприходование по факту слива (не по ТТН)
- Бухгалтерский: потери <0.5% — за счёт поставщика (не списываем)
```

```
  - Налоговый: ретро-бонус учесть при расчёте себестоимости
  - Оптимизация: отсрочка 14 дней — контролировать сроки оплаты,
    при просрочке — пеня, отразить в расходах
```

---

3. Код vs AI: разграничение ответственности

Принцип

Не всё должно проходить через AI. ClearLedger использует **код для детерминированных операций** и **AI для интерпретации и решений**.

- **Код** (Python): быстро (<50мс), бесплатно (0₽), предсказуемо — но жёсткий
- **AI** (Skills): гибкий, понимает контекст, предлагает решения — но 1-3 сек и стоит токены

Золотое правило: если результат вычисляется по формуле — это код. Если нужно суждение — это AI.

Матрица: что делает код, что делает AI

Операция	Код	AI	Почему
Приём JSON из STS API	✓		Фиксированный формат, тысячи запросов/день
Парсинг CSV банковского реестра	✓		Формат известен, детерминированный парсинг
OCR PDF товарной накладной		✓	Неструктурированный документ, разные форматы
Расчёт суммы, НДС, итогов	✓		Чистая математика: amount × rate
Pydantic-валидация JSONB	✓		Схема известна, проверка по правилам
Сверка: сумма A == сумма B	✓		Арифметическое сравнение, ноль интерпретации
Анализ **причины** расхождения		✓	Нужен контекст: история, паттерны, связи
Генерация XML для 1С	✓		Шаблон + данные, детерминированный результат
Выбор учётного отражения		✓	Зависит от договора, контура, оптимизации
Статусная машина документов	✓		draft→validated→verified — конечный автомат

Операция	Код	AI	Почему
Построение Document Graph	✓		Связи по известным правилам (ТТН→резервуар)
Поиск аномалий в истории		✓	Паттерны, сезонность, нетипичные отклонения
Файловый обмен с 1С	✓		Polling папки, перемещение файлов
Интерпретация условий договора		✓	Естественный язык → бизнес-правила
Предложение корректировок		✓	Суждение: что исправить и почему
Event Bus / воркеры	✓		Инфраструктура, маршрутизация событий
Прогнозы (точка заказа)		✓	Анализ трендов, сезонность, внешние факторы

Гибридные операции: код + AI

Самые частые операции — **гибридные**: код делает тяжёлую работу, AI подключается при необходимости.

```
Пример: Сверка кассы со сменным отчётом

Код (мгновенно, 0₽):
  1. Загрузить оба документа из БД
  2. cash_sum_receipts − cash_sum_shift = delta
  3. bank_sum_receipts − bank_sum_shift = delta
  4. Если |delta| < порог → ✓ ОК, записать результат
  5. Если |delta| > порог → вызвать AI

AI (1–3 сек, $0.0004):
  6. Загрузить навык reconciliation/cash_vs_receipts.md
  7. Загрузить связанные документы по графу
  8. Проанализировать: почему расхождение?
  9. Предложить корректировку с уверенностью
```

```
Пример: Нормализация данных из STS API

Код (мгновенно, 0₽):
  1. Получить JSON из STS API
  2. Валидировать через Pydantic
  3. Маппинг полей → структура documents
  4. Рассчитать производные (total_amount, vat_amount)
  5. Сохранить в documents со статусом draft
```

```
  6. Построить связи в document_links

AI (только при ошибке):
  7. Pydantic-валидация упала → AI анализирует что изменилось в формате
  8. Появились неизвестные поля → AI предлагает маппинг
```

Экономика: зачем разграничивать

```
Без разграничения (всё через AI):
  6000 смен × 7 сверок × $0.0004 = $16.8/месяц (~1 500₽)
  Задержка: 1-3 секунды на каждую операцию

С разграничением (код + AI по необходимости):
  Код: 6000 × 7 = 42 000 сверок → мгновенно, 0₽
  AI: ~5% расхождений = 2 100 вызовов → $0.84/месяц (~75₽)
  Задержка: <50мс для 95% операций

Экономия: 20× дешевле, 60× быстрее для типовых случаев
```

Правило для разработчика

При добавлении новой функциональности:

1. **Можно описать как формулу или if/else?** → Код
2. **Нужен контекст, суждение, интерпретация?** → AI (навык)
3. **Код справляется в 95%, но 5% нестандартных?** → Гибрид: код + AI fallback
4. **Формат данных может измениться?** → Код для текущего + AI для обработки изменений

---

4. Онбординг: вход в работающий бизнес

Проблема

ClearLedger не запускается на зелёном поле. Мы приходим в бизнес, который уже работает:

- Есть свои Excel-таблицы, 1С, учётные системы
- Есть сложившиеся процессы (часто неоптимальные, но привычные)
- Есть люди, которые «знают как тут всё устроено»
- Есть накопленные данные за месяцы и годы

Классический подход: консультант изучает бизнес 2-4 недели, пишет ТЗ, разработчик настраивает систему. Долго, дорого, субъективно.

Подход ClearLedger: AI сам исследует существующий учёт и генерирует набор навыков под конкретный бизнес.

Фаза 1: Discovery (AI-исследование)

AI получает доступ к существующим данным и анализирует:

```
1. Источники данных
   - Какие системы используются (кассы, банки, склад, CRM)
   - В каких форматах данные (API, CSV, Excel, PDF, 1C)
   - Как часто обновляются
   - Какие поля есть, что они значат

2. Процессы учёта
   - Как закрывается смена / день / месяц
   - Кто с кем сверяется и что сверяет
   - Какие расхождения типичны и как их решают
   - Где ручная работа, которую можно автоматизировать

3. Договорная база
   - Условия поставщиков (допуски, бонусы, сроки)
   - Условия клиентов (рассрочки, скидки)
   - Налоговые особенности (спецрежимы, льготы)

4. Болевые точки
   - Где чаще всего ошибки
   - Что занимает больше всего времени
   - За что были штрафы или претензии
```

Фаза 2: Генерация навыков

На основе Discovery AI создаёт **черновики навыков** под конкретный бизнес:

```
Пример: Сеть АЗС «НефтоЛига» (12 станций)

Discovery выявил:
  - STS API v2 (стандартный формат)
  - Банк: Сбер, реестр раз в сутки, формат CSV с особенностями
  - Топл. карты: TradeCorp + собственная система «НЛ-карт»
  - Потери: у станций 3, 7, 11 систематическое превышение нормы
  - Договор поставки: ретро-бонус 1.5% при >50 тонн/месяц
  - 1С: Бухгалтерия 3.0, учётная политика — средневзвешенная
```

```
AI сгенерировал навыки:
  skills/
  ├── sources/
  │   ├── sts_v2_neftoliga.md          # STS + их конфигурация
  │   ├── sber_acquiring_neftoliga.md   # Формат Сбера с их особенностями
  │   └── nl_cards.md                   # Их собственная карточная система
  ├── reconciliation/
  │   ├── problem_stations_3_7_11.md    # Усиленный контроль потерь
  │   └── sber_delay_compensation.md    # Учёт задержки реестра на 1 день
  ├── accounting/
  │   └── retro_bonus_neftoliga.md      # Ретро-бонус по их договору
  └── contracts/
      └── supply_neftosnab.md           # Условия их поставщика
```

Фаза 3: Валидация человеком (жёсткая)

Реальность: Данные клиентов — это ад. CSV в CP1251 вперемешку с UTF-8. PDF — сканы рукописных бумажек. Excel с объединёнными ячейками и формулами, ссылающимися на удалённые листы. AI может галлюцинировать при генерации навыков на грязных данных.

Поэтому Фаза 3 — не формальность, а жёсткий контроль:

1. **Визуальный редактор навыков:** менеджер видит, какой кусок исходных данных AI превратил в правило. Не «чёрный ящик», а прозрачная цепочка: исходные данные → интерпретация AI → навык

```
┌──────────────────────────────────────────────────────────┐
│ Навык: problem_stations_3_7_11.md                        │
│                                                          │
│ Источник: Сменные отчёты за 01.2026-02.2026              │
│ AI обнаружил: потери 0.8-1.2% на станциях 3, 7, 11       │
│                                                          │
│ ┌─ Правило (редактируемое) ──────────────────────┐       │
│ │ Порог warning: [0.3%] ← стандарт: 0.5%         │       │
│ │ Порог critical: [0.7%] ← стандарт: 1.0%        │       │
│ │ Действие: уведомить менеджера                  │       │
│ └────────────────────────────────────────────────┘       │
│                                                          │
│ Основание (данные AI):                                   │
│   Ст.3: янв 0.9%, фев 1.1%                               │
│   Ст.7: янв 0.8%, фев 0.85%                              │
│   Ст.11: янв 1.2%, фев 1.15%                             │
│                                                          │
```

```
│ [✓ Утвердить] [✎ Скорректировать] [✗ Отклонить]     │
└──────────────────────────────────────────────────────┘
```

2. **Менеджер корректирует** на основе знания бизнеса:

```
Менеджер: "Для станции 11 причина — старые резервуары,
           пороги 0.5% / 1.0%. Остальные — согласен."

AI: обновляет навык, фиксирует решение и причину.
```

3. **Обязательная проверка на реальных данных:** после утверждения навыка — прогон на данных за последний месяц. Менеджер видит: «вот что бы ClearLedger сделал». Если результат неадекватный — возврат на редактирование.
4. **Защита от галлюцинаций AI:** каждый навык фиксирует не только правило, но и **основание** (конкретные данные и документы, из которых AI сделал вывод). Если основание неверное — навык отклоняется.

Фаза 4: Параллельная работа

ClearLedger работает **рядом** с существующим учётом 2-4 недели:

```
Неделя 1-2: Наблюдение
  ClearLedger забирает данные, сверяет, формирует протоколы
  НЕ выгружает в 1С — только показывает результаты
  Менеджер сравнивает: "раньше я 2 часа искал, а тут 3 секунды"

Неделя 3-4: Калибровка
  AI корректирует навыки на основе реальных данных
  Точность сверки: 85% → 92% → 97%
  Менеджер подтверждает или поправляет предложения AI
```

Фаза 5: Переключение

```
Когда точность >95% и менеджер уверен:
  1. ClearLedger начинает формировать документы для 1С
  2. Первые 1-2 недели — бухгалтер проверяет каждый документ
  3. Потом — выборочная проверка
  4. Потом — только исключения (расхождения, новые ситуации)
```

Эффект масштаба: каждый клиент делает платформу умнее

```
Клиент 1 (АЗС «НефтоЛига»):
  Онбординг: 4–6 недель
  Создано навыков: 12
  Из них переиспользуемых: 8 (STS API, Сбер, базовые сверки)

Клиент 2 (АЗС «ТопливоПлюс», тоже STS + Сбер):
  Онбординг: 2–3 недели (8 навыков уже есть)
  Новых навыков: 4 (только их специфика)
  Из них переиспользуемых: 2

Клиент 10:
  Онбординг: 1–2 недели
  Библиотека навыков покрывает 80% типовых ситуаций
  AI генерирует только навыки под уникальные особенности
```

Каждый новый клиент обогащает библиотеку навыков. AI становится опытнее — как консультант, который видел десятки бизнесов в отрасли.

---

5. Три контура учёта

Одни данные — три представления

Каждая операция отражается в трёх контурах одновременно, но по-разному:

```
Первичный документ (сменный отчёт, ТТН, акт)
         │
         ├──→ Управленческий учёт
         │      Цель: реальная картина для решений
         │      Правила: экономическая суть операции
         │      Результат: маржа, потери в ₽, KPI, прогнозы
         │
         ├──→ Бухгалтерский учёт
         │      Цель: официальная отчётность (ПБУ/ФСБУ)
         │      Правила: стандарты бухучёта
         │      Результат: документы для 1С → баланс, ОПиУ
         │
         └──→ Налоговый учёт
                Цель: корректный расчёт налогов
                Правила: НК РФ
                Результат: подсветка вариантов + рекомендации → решение
специалиста
```

Где контуры расходятся (примеры)

Операция	Управленческий	Бухгалтерский	Налоговый
Потери ГСМ 0.3%	Потери 0.3% → сигнал менеджеру	Дт 44 Кт 41 (в пределах норм)	Уменьшает базу (НК ст.254 п.7)
Потери ГСМ 1.5%	Потери 1.5% → расследование	0.5% → Дт 44, 1.0% → Дт 94	0.5% уменьшает базу, 1.0% — НЕТ
Ретро-бонус от поставщика	Снижение себестоимости	Дт 60 Кт 41 (корректировка)	Внереализационный доход
Бесплатная раздача (промо)	Маркетинговый расход	Дт 44 Кт 41 + начислить НДС	НДС начисляется на рыночную стоимость
Аренда АЗС	Ежемесячный расход	По ФСБУ 25 — право пользования	Линейно по сроку договора

Как AI подсвечивает варианты отражения

```
Шаг 1: AI читает первичный документ (факт)
        "Поступило 5000л АИ-92, по ТТН 5025л, недостача 25л"

Шаг 2: AI загружает навык учёта (accounting/fuel_receipt.md)
        "Потери при транспортировке — проверить норму"

Шаг 3: AI загружает условия договора (contracts/supply_contract.md)
        "Допустимая недостача: 0.5% от массы"

Шаг 4: AI рассчитывает
        25л / 5025л = 0.497% < 0.5% → в пределах допустимого по договору

Шаг 5: AI формирует рекомендации для каждого контура
        Управленческий: потери 25л × 52.50₽ = 1 312.50₽ (за счёт поставщика)
        Бухгалтерский: оприходовать 5000л, недостачу — претензия поставщику
        Налоговый: НДС принять к вычету только на 5000л (фактическое кол-во)

Шаг 6: AI подсвечивает риск
        "Внимание: если не предъявить претензию поставщику на 1 312.50₽,
         потери ляжут на нас и НЕ уменьшат налоговую базу
         (сверхнорма без претензии). Рекомендация: предъявить."
        → Решение принимает менеджер/бухгалтер, AI фиксирует основание
```

Реализация контуров

Управленческий — целиком внутри ClearLedger:

- Materialized views, сводные таблицы, дашборды
- AI-аналитика по запросу ("сравни маржу по станциям за квартал")
- Прогнозы (заказ топлива, планирование инкассации)

Бухгалтерский — ClearLedger готовит документы → 1С проводит:

- XML-документы: продажи, поступления, списания, инкассация
- 1С формирует проводки по своей учётной политике
- ClearLedger контролирует: ожидаемое vs фактическое из 1С

Налоговый — ClearLedger подсвечивает варианты → специалист принимает решение → 1С формирует декларации:

- AI подсвечивает варианты квалификации операций с указанием НПА
- Раздельный учёт НДС (если есть облагаемые и необлагаемые операции) — подсказка
- Нормативные потери: расчёт по Минэнерго №527, ГОСТ Р 8.595-2004
- Контроль: все ли вычеты НДС по поступлениям приняты
- **Решение фиксируется** с указанием: кто утвердил, на каком основании, версия навыка

Юридический контур налоговых рекомендаций

Принцип: AI не принимает налоговое решение, AI только предлагает варианты с рисками.

Фиксируем в карточке решения:

- тип вопроса (НДС/налог на прибыль/акцизы);
- вариант A/B и риск-оценка;
- основание (НПА, письмо, договор, судебная практика при наличии);
- кто утвердил (ФИО, роль, дата/время);
- юридическая формулировка: "Рекомендация AI не является налоговой консультацией".

Шаблон согласования спорной трактовки:

```
Issue: TAX-2026-02-024
Операция: Потери ГСМ сверх нормы
Вариант AI: Не учитывать в расходах
Альтернатива: Учитывать частично при наличии претензии поставщику
Риск: Средний
Основание: НК РФ ст.252, ст.254; договор №...
```

```
Решение специалиста: [выбранный вариант]
Подпись: [ФИО, должность, дата]
```

---

6. Хранение данных

PostgreSQL + JSONB

Решение: PostgreSQL 16, гибридная модель.

Почему PostgreSQL, а не MongoDB:

- Транзакции (ACID) — критично для учёта, нельзя потерять данные
- SQL + JSONB в одной БД — не нужно два хранилища
- GIN-индексы на JSONB — быстрый поиск внутри JSON
- Партиционирование — архивация старых периодов

Правило разделения: колонка vs JSONB:

В колонку	В JSONB metadata
Участвует в WHERE, GROUP BY, ORDER BY	Специфично для типа документа
Нужен внешний ключ (FK)	Вложенные структуры
Участвует в сверке	Редко запрашивается отдельно
Нужен уникальный индекс	Структура может меняться

```sql
-- Пример: общие поля — колонки, специфика — JSONB
CREATE TABLE documents (
    id              UUID PRIMARY KEY,
    raw_id          UUID REFERENCES raw_data(id),
    doc_type        TEXT NOT NULL,          -- колонка: фильтр
    doc_date        TIMESTAMPTZ NOT NULL,   -- колонка: диапазон
    station_id      UUID,                   -- колонка: FK
    status          TEXT NOT NULL,          -- колонка: фильтр
    metadata        JSONB NOT NULL,         -- JSONB: всё остальное
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Канонические типы документов (ГСМ)

Таблица `documents` — не безразмерный JSON. Каждый `doc_type` имеет фиксированную Pydantic-модель с обязательными полями:

doc_type	Назначение	Обязательные поля в metadata
`shift_report`	Сменный отчёт	shift_number, operator, period_from, period_to, products[{code, qty, amount}], payments[{type, amount}], tank_readings[{tank_id, level, volume, temp}]
`transaction`	Транзакция отпуска	trk_id, nozzle, fuel_code, qty, amount, payment_type, timestamp
`acquiring_register`	Реестр эквайринга	bank_name, terminal_id, operations[{amount, card_mask, rrn, timestamp}], total_amount
`fuel_card_ops`	Операции по топл. картам	card_number, contractor_code, operations[{fuel_code, qty, amount, timestamp}]
`online_order`	Онлайн-заказ (MSTO)	order_id, fuel_code, qty, amount, trk_id, status, payment_method
`delivery_ttn`	ТТН (поступление)	ttn_number, supplier_code, fuel_code, doc_volume, doc_mass, fact_volume, fact_mass, density, temperature
`tank_reading`	Замер резервуара	tank_id, fuel_code, volume, mass, level, temperature, water_level, timestamp

Каждый тип валидируется Pydantic-моделью при создании документа. Новое поле можно добавить в JSONB, но обязательные поля — контракт, который нельзя нарушить.

Если поле из JSONB начинает использоваться в >30% запросов — выносим в Generated Column:

```
ALTER TABLE documents ADD COLUMN total_amount NUMERIC
    GENERATED ALWAYS AS ((metadata->>'total_amount')::numeric) STORED;
```

Индексация

```
-- B-tree: для фильтров и диапазонов
CREATE INDEX idx_docs_type_date ON documents (doc_type, doc_date);
CREATE INDEX idx_docs_station   ON documents (station_id, doc_date);
CREATE INDEX idx_docs_status    ON documents (status) WHERE status !=
'verified';

-- GIN: для поиска внутри JSONB (только слой 2, не RAW)
```

```
CREATE INDEX idx_docs_metadata ON documents USING GIN (metadata
jsonb_path_ops);
```

Почему GIN только на documents, не на raw_data:

- GIN замедляет INSERT на 20-40%
- raw_data — write-heavy (пишем постоянно, ищем редко)
- documents — read-heavy (ищем часто, пишем реже)

Партиционирование

Решение: Месячные партиции по doc_date.

Почему: При 100 АЗС × 2 смены = ~73K документов/год. Месячная партиция = ~6K строк — мгновенные запросы. Главная польза — архивация: `DETACH PARTITION` отцепляет старый месяц без блокировки таблицы.

Валидация JSONB

Решение: Двухуровневая — Pydantic в Python + CHECK в PostgreSQL.

Почему два уровня:

- Pydantic — полная валидация с понятными ошибками, автодокументация API
- CHECK в БД — страховка от мусора при прямом INSERT (миграции, скрипты)

Качество входных данных и fallback-процесс

Для каждого входного файла/пакета считаем `input_quality_score` (0-100):

- валидность структуры;
- полнота обязательных полей;
- корректность кодировок/типов;
- доля строк, прошедших автопарсинг без ручных правок.

Маршрутизация по качеству:

- `score >= 90` : автозагрузка в pipeline;
- `70–89` : автозагрузка с warning + очередь на выборочную проверку;
- `<70` : стоп автопроведения, ручная валидация в UI.

Fallback-процесс (ручная валидация):

1. Документ уходит в очередь `manual_validation` .

2. Оператор исправляет поля через UI-редактор с подсветкой проблем.
3. Повторная валидация Pydantic + CHECK.
4. Только после успешной проверки документ возвращается в pipeline.

Обслуживание: VACUUM и write amplification

Проблема: Частые обновления документов (статусы, корректировки) + большие JSONB → PostgreSQL создаёт мёртвые кортежи (dead tuples), база раздувается.

Решение:

- **autovacuum** с агрессивными настройками для таблиц documents и document_versions:

```
autovacuum_vacuum_scale_factor = 0.05   (вместо 0.2)
autovacuum_analyze_scale_factor = 0.02  (вместо 0.1)
```

- **TOAST** для больших JSONB: PostgreSQL автоматически выносит JSONB >2KB в отдельное хранилище. Обновление колонки status не трогает JSONB.
- **Мониторинг:** pg_stat_user_tables → n_dead_tup. Алерт если dead_tup > 20% от n_live_tup
- **Мониторинг TOAST:** ежедневный контроль размера toast-таблиц documents/document_versions (`pg_total_relation_size`). Алерт при росте >15% за неделю без роста объёма документов.
- **Партиции помогают:** VACUUM работает на уровне партиции (месяц), а не всей таблицы. Старые партиции (заархивированные) не обновляются → не раздуваются.
- **Ретенция и архив:** партиции старше 12 месяцев `DETACH` → архив в S3-совместимое хранилище (Parquet + checksum + manifest), в production остаются последние 12 месяцев.
- **Правило:** Не обновлять JSONB metadata целиком при смене статуса. Статус — отдельная колонка, JSONB трогаем только при реальном изменении данных.

---

7. Сверка данных

Гибридный подход: код + навыки

Арифметика сверки — **код** (Python): загрузить два документа, посчитать разницу, сравнить с порогом. Мгновенно, бесплатно, 95% случаев закрывается без AI.

Если расхождение превышает порог — подключается **AI** с навыком из `skills/reconciliation/*.md` : анализирует причину по Document Graph, предлагает корректировку с указанием уверенности.

Пороги (ok / warning / critical) хранятся в БД — менеджер меняет без разработчика и без релиза.

Матрица сверок (ГСМ)

Сверка	Что сравнивается	Порог warning	Порог critical
Операции ↔ Сменный отчёт	Литры/₽ по виду топлива	>0.1%	>1%
Касса ↔ Сменный отчёт	₽ наличные + безнал	>0 ₽	>100 ₽
Банк ↔ ККТ	₽ безнал (эквайринг)	>0 ₽	>100 ₽
Топл.карты ↔ Операции	Поштучно: литры, ₽, время ±5мин	1 непарная	>3 непарных
Онлайн ↔ Операции	Поштучно: литры, ₽, ТРК	1 непарная	>3 непарных
ТТН ↔ Факт приёмки	Масса (кг), объём (л)	>0.3%	>0.5%
Остатки резервуаров	Факт vs расчёт (было+приход−расход)	>0.3%	>0.5%

Пороги хранятся в БД/Supabase — менеджер меняет без разработчика.

Workflow корректировок

```
Расхождение → AI анализирует → Уверенность?
  >95%  → Автокорректировка + запись в лог (менеджер видит)
  70-95% → Предложение менеджеру (approve/reject одним кликом)
  <70%  → Ручной разбор менеджером
```

Все корректировки логируются: field_path, old_value, new_value, reason, who, when. Иммутабельный audit trail.

Роли и полномочия (RBAC)

Кто что может — определяется сразу, а не «потом допилим»:

Роль	Видит	Может
Оператор АЗС	Свою станцию	Подтвердить приёмку, закрыть смену

Роль	Видит	Может
Менеджер	Станции своего региона	Approve/reject корректировку AI, подтвердить сверку
Руководитель сети	Всю сеть	Утвердить спорные корректировки, полная аналитика
Бухгалтер	Выверенные документы	Запустить выгрузку в 1С, видеть статус проведения
Администратор	Всё	Управление навыками, порогами, ролями, настройка 1С-маппинга

Матрица подтверждений:

Действие	Кто подтверждает	Принцип
AI-автокорректировка (>95%)	Автоматически	Менеджер видит в логе, может отменить
AI-предложение (70-95%)	Менеджер	Approve/reject одним кликом
Ручная корректировка суммы	Менеджер + руководитель	Принцип «4 глаза»
Выгрузка в 1С	Бухгалтер	Только verified документы
Изменение порогов сверки	Руководитель	Аудит изменений
Создание/изменение навыков	Администратор	Версионирование навыков
Корректировка закрытого периода	Руководитель + бухгалтер	Обязательное обоснование

Правило: чем дороже ошибка — тем выше уровень подтверждения.

Реализация: Таблица `permissions` (role, action, scope). Scope = конкретные станции/регион/вся сеть. Проверка на уровне API middleware — не UI, а backend. Невозможно обойти через прямой запрос.

Машина времени: история документа

Клиенту недостаточно знать что «логи есть». Нужен **интерфейс**, где видно как жил документ:

```
Документ: Смена №6890, АЗС 208, 20.02.2026

v1  07:31  [код]        draft
```

```
    Создан из STS API. Итого: 154 646.00₽

v2  07:32  [код]        validated
    Автосверка: касса ✓, банк ✓, топл.карты ✓

v3  09:15  [AI Haiku]   validated + warning
    Расхождение: эквайринг −200₽
    AI: "Банковский реестр задержан, ожидать до 12:00"

v4  12:05  [AI Haiku]   validated
    Реестр получен. bank_sum: 78 000 → 78 200. Δ = 0₽

v5  14:00  [менеджер]   verified
    Подтверждено Петровой А.В. Готово к выгрузке.

v6  15:30  [код]        exported
    XML → D:\Exchange\to_1c\inbox\msg-20260220-001.xml

v7  16:14  [1C]         posted
    Проведён. Номер РП-000123.
```

Diff-режим: клик на любые две версии → визуальное сравнение:

```
  bank_sum:   78 000.00 → 78 200.00  [AI, банковский реестр]
  status:     validated → verified    [менеджер Петрова, ручное]
```

Реализация: Таблица `document_versions`:

```
CREATE TABLE document_versions (
    id          UUID PRIMARY KEY,
    document_id UUID REFERENCES documents(id),
    version     INT NOT NULL,
    actor       TEXT NOT NULL,          -- 'system', 'ai:haiku',
'user:petrova'
    actor_type  TEXT NOT NULL,          -- 'code', 'ai', 'human'
    changes     JSONB NOT NULL,         -- [{field, old, new, reason}]
    snapshot    JSONB NOT NULL,         -- полный снимок документа
    created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Snapshot — полная копия документа на момент версии. Дорого по месту, но дёшево по надёжности: всегда можно восстановить любое состояние без пересчёта дельт.

---

8. Document Graph

Решение: Таблица `document_links` — явные связи между документами.

Почему не просто FK:

- AI-агенту нужен граф, чтобы **находить причины** расхождений, а не просто факты
- Пример: расхождение по кассе → AI проходит по графу → чек отменён → возврат не прошёл через банк → вот причина
- Граф строится автоматически при парсинге (ТТН → Резервуар → Смена → Чеки → Банк)

```
CREATE TABLE document_links (
    source_id   UUID REFERENCES documents(id),
    target_id   UUID REFERENCES documents(id),
    link_type   TEXT NOT NULL,  -- 'receipt_to_tank', 'shift_to_check', ...
    metadata    JSONB DEFAULT '{}',
    PRIMARY KEY (source_id, target_id, link_type)
);
```

Периодический relinking Document Graph

После обновлений навыков связи могут устаревать. Поэтому запускаем ретроспективный relinking:

- nightly-job для документов последних 30 дней;
- weekly-job для последних 6 месяцев;
- пересобираем только связи, затронутые изменёнными навыками/маппингами;
- сравниваем старый и новый граф, фиксируем diff в audit trail.

Цель: не допустить деградации причинно-следственных цепочек при эволюции навыков.

---

9. Интеграция с 1С

Транспорт: файловый обмен

Решение: XML-файлы в общую папку. 1С забирает регламентным заданием.

Почему файловый, а не HTTP API:

- Работает при выключенной/перезагружающейся 1С — файлы копятся в папке
- Не нужно публиковать базу 1С в сеть (безопасность)

- Встроенная очередь — файловая система
- Простота отладки — XML можно открыть и посмотреть

Структура папок:

```
D:\Exchange\ClearLedger_1C\
├── to_1c\inbox\        ← ClearLedger кладёт XML
├── to_1c\done\         ← 1C переносит обработанные
├── to_1c\error\        ← 1C переносит с ошибками
├── from_1c\inbox\      ← 1C кладёт квитанции (feedback)
└── from_1c\done\       ← ClearLedger забирает
```

Формат: Custom XML (не CommerceML)

Почему не CommerceML:

- CommerceML не покрывает специфику ГСМ (плотность, температура, ТРК, резервуары)
- CommerceML заточен под каталоги товаров и простые продажи
- Custom XML — полная свобода структуры, обработку в 1С пишем один раз

Почему не передаём проводки:

- 1С сама формирует проводки при проведении документов — это её работа
- У 1С есть учётная политика, регистр счетов учёта номенклатуры — всё настроено
- Если мы передадим проводки, а учётная политика изменится — наши проводки устареют, а 1С не пересчитает
- Мы передаём **факты** (продали, приняли, списали), 1С решает **как отразить в учёте**

Пример XML (отчёт о розничных продажах):

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ClearLedger version="1.0" message_id="msg-20260224-001">
  <Document type="RetailSalesReport" id="uuid-001">
    <Station code="208" name="АЗС 208"/>
    <Shift number="6890"/>
    <Period from="2026-02-24T07:31:00" to="2026-02-24T19:45:00"/>
    <Operator>Петрова А.В.</Operator>

    <Products>
      <Product code="AI92K5" name="Бензин АИ-92-К5"
               qty="1240.000" unit="л"
               price="52.50" amount="65100.00"
               vat_rate="20" vat_amount="10850.00"/>
```

```
      <Product code="AI95K5" name="Бензин АИ-95-К5"
               qty="890.000" unit="л"
               price="56.80" amount="50552.00"
               vat_rate="20" vat_amount="8425.33"/>
      <Product code="DTK5" name="Дизельное топливо ЕВРО-К5"
               qty="670.000" unit="л"
               price="58.20" amount="38994.00"
               vat_rate="20" vat_amount="6499.00"/>
    </Products>

    <Payments>
      <Payment type="cash" amount="42300.00"/>
      <Payment type="card" amount="78200.00"/>
      <Payment type="fuel_card" amount="24146.00"
               counterparty_code="LOGISTIC-001"/>
      <Payment type="online" amount="10000.00"/>
    </Payments>
  </Document>
</ClearLedger>
```

Обратная связь (feedback):

```
<ClearLedgerFeedback message_id="msg-20260224-001">
  <Result doc_id="uuid-001" status="posted"
          onec_number="РП-000123" onec_guid="e1cib-..."
          posted_at="2026-02-24T20:14:55"/>
</ClearLedgerFeedback>
```

Справочники: mapping по кодам

Решение: Таблица соответствий (code в ClearLedger ↔ code в 1С).

Почему по кодам, а не по наименованиям:

- В ClearLedger: "Бензин АИ-92", в 1С: "АИ-92 Бензин К5 ГОСТ" — наименования не совпадут
- Код `AI92K5` — уникальный, не меняется при переименовании

Реальность: нетиповые 1С и EPF-загрузчик

Проблема: «Типовых» 1С в сетях АЗС почти не бывает. У каждого клиента — допиленная конфигурация с обязательными полями, которых нет в стандарте (аналитика «Подразделение затрат», «Объект строительства», кастомные регистры).

Решение: ClearLedger поставляет не только XML, но и **EPF-обработку загрузки** на стороне 1С:

```
ClearLedger                                  1С клиента

                                             ┌──────────────────────┐
XML → папка to_1c/inbox/ ──────────────→     │ EPF-загрузчик        │
                                             │  ├─ Читает XML       │
                                             │  ├─ Маппинг полей    │
                                             │  │  (настраивается)  │
                                             │  ├─ Заполняет доп.   │
                                             │  │  реквизиты 1С     │
                                             │  ├─ Создаёт документ│
                                             │  └─ Проводит         │
                                             └──────────────────────┘
```

EPF-загрузчик:

- Написан на 1С, поставляется как внешняя обработка (epf-файл)
- **Настраиваемый маппинг:** таблица соответствий (поле XML → реквизит 1С). Настраивает 1С-специалист при внедрении, не разработчик ClearLedger
- **Обязательные поля 1С:** загрузчик знает, какие доп. реквизиты нужны в конкретной базе, и заполняет их из маппинга или справочников 1С
- **Валидация перед проведением:** если документ не проходит проверку 1С — EPF пишет ошибку в feedback, не теряет данные
- **Версионируется** вместе с конфигурацией клиента

Стандартные пресеты:

- **Бухгалтерия 3.0** (~80% клиентов) — пресет «из коробки», минимальная настройка
- **Бухгалтерия 3.0 + расширения** — пресет + ручная донастройка 1С-ником (2-4 часа)
- **Нетиповая конфигурация** — полностью ручная настройка маппинга 1С-специалистом

Процедура внедрения в нетиповую 1С (sandbox + SLA)

1. Поднимаем sandbox-копию базы 1С клиента (обезличенные данные).
2. Прогоняем smoke-пакет XML (типовые документы + edge-cases).
3. Фиксируем gap-list обязательных реквизитов/проверок в базе клиента.
4. 1С-специалист настраивает маппинг и EPF до зелёного прогона.
5. Подписываем протокол совместимости версии XML/EPF и запускаем в production.

SLA на донастройку 1С:

Приоритет	Сценарий	SLA
P1	Блокирующая ошибка проведения в 1С	реакция 2 часа, workaround 8 часов
P2	Частичная несовместимость реквизитов	реакция 1 рабочий день, исправление 2 дня
P3	Некритичная оптимизация маппинга	в плановый релиз (до 5 рабочих дней)

Важно: AI-Discovery НЕ анализирует внутреннюю структуру 1С. Таблицы 1С (_Reference123, _Document456) — проприетарные, метаданные в BLOBах, без COM/OData с полными правами AI ничего не поймёт. AI-Discovery исследует **бизнес-процессы клиента** (источники данных, форматы, сверки), а настройку EPF делает 1С-специалист по стандартному пресету.

Версионирование XML-схемы и EPF

Проблема: ClearLedger обновился (новое поле в XML), клиент не обновил EPF — выгрузка падает.

Решение: Обязательная версия схемы в каждом XML:

```
<ClearLedger version="1.2" schema_min="1.0" message_id="msg-...">
```

EPF при загрузке проверяет:

```
Если version > моя_макс_версия:
  → Ошибка: "EPF поддерживает версии 1.0-1.1. Получена 1.2.
     Обновите обработку загрузки."
  → Файл → папка error/ (не теряется)
  → Уведомление администратору ClearLedger
```

Правила совместимости:

- **Minor version** (1.0 → 1.1): новые необязательные поля. Старый EPF игнорирует их — работает
- **Major version** (1.x → 2.0): изменена структура. Старый EPF НЕ работает — принудительное обновление
- ClearLedger отправляет `schema_min` — минимальная версия EPF, которая сможет обработать файл

Заднее число: корректировки после закрытия периода

Проблема: Сверка прошла, документ в 1С, месяц закрыт (Дата запрета изменения данных). Через 2 недели банк прислал корректировку эквайринга за февраль.

Решение: корректирующие документы, а не правка задним числом.

```
Ситуация:
  Февраль закрыт в 1С (дата запрета: 28.02.2026)
  15 марта банк прислал исправленный реестр за 20 февраля
  Разница: +350₽ (была отмена, банк не передал вовремя)

Что делает ClearLedger:
  1. Фиксирует расхождение в документе февраля (новая версия)
  2. НЕ пытается изменить февральский документ в 1С
  3. Формирует КОРРЕКТИРУЮЩИЙ документ с датой марта:

     <Document type="Correction" id="uuid-corr-001">
       <OriginalDoc id="uuid-001" date="2026-02-20"/>
       <CorrectionDate>2026-03-15</CorrectionDate>
       <Reason>Корректировка банковского реестра эквайринга</Reason>
       <Changes>
         <Change field="bank_amount" old="78200.00" new="78550.00"/>
       </Changes>
     </Document>

  4. 1С проводит корректировку мартом (февраль не трогаем)
```

Три типа корректировок:

Тип	Когда	Документ для 1С
Период открыт	Текущий месяц	Исправляем оригинальный документ
Период закрыт, сумма < порога	Следующий открытый месяц	Корректирующий документ
Период закрыт, сумма > порога	Требует утверждения руководителя	Бухгалтерская справка + корректировка

ClearLedger автоматически проверяет дату запрета 1С перед отправкой. Если документ попадает в закрытый период — формирует корректировку в текущем периоде. Порог суммы для автокорректировки настраивается (по умолчанию: до 1000₽ — автоматически, выше — утверждение).

Типичные ловушки

Ловушка	Решение
Справочники не совпадают	Mapping по кодам, синхронизация перед выгрузкой
Округление НДС	Считать построчно, итог = сумма строк
Кодировка XML	UTF-8 с BOM
Нумерация документов	Передаём как "внешний номер", 1С присваивает свой
Закрытый период в 1С	Автопроверка даты запрета → корректирующий документ
Литры vs кг (ТТН)	Хранить оба + плотность + температуру
Нетиповая конфигурация 1С	EPF-загрузчик с настраиваемым маппингом
Корректировка банка «задним числом»	Корректирующий документ в текущем периоде

---

10. AI-агенты

Три уровня участия

Уровень	Что делает	Когда включаем
Наблюдатель	Аналитика, отчёты, поиск аномалий (read-only)	MVP
Советник	Предлагает корректировки, менеджер подтверждает	v1.0
Исполнитель	Автокорректировка типовых (уверенность >95%)	v1.0+ (после накопления истории)

Semantic API

Решение: Высокоуровневые endpoints + SQL-доступ к read-only реплике.

Почему оба:

- Semantic API — для типовых задач (найди смены с расхождениями) — быстро, мало токенов

- SQL-доступ — для нестандартных запросов (AI пишет SQL сам) — гибко, любой анализ
- Ограничения SQL: только SELECT, timeout 30s, max 10K rows

Dynamic Schema Context

Решение: Endpoint `/api/v1/schema/context?task=reconciliation` — возвращает минимальный контекст для AI.

Почему: Полная схема БД = 5000 токенов. Для задачи сверки нужно 500 токенов. Экономия = скорость + дешевле.

Prompt Caching: метрики и TTL-политика

Кэш считаем частью себестоимости и управляем как продуктовой метрикой:

- `cache_hit_rate` по типам контекста (договоры, навыки, история);
- `cache_miss_cost` (руб/день);
- `avg_prompt_tokens` до/после кэша;
- `p95_latency` AI-запросов.

TTL-политика (стартовые значения):

- договоры: 30 минут (редко меняются);
- навыки: 10 минут (меняются чаще при активной калибровке);
- исторические паттерны: 5 минут.

Пересмотр TTL еженедельно по hit/miss: цель — `cache_hit_rate > 70%` на повторяемых задачах.

Самообучение (v1.0+)

AI анализирует историю корректировок и:

- Предлагает новые навыки или обновления существующих (паттерн → навык)
- Корректирует пороги warning/critical
- Увеличивает долю автокорректировок с ростом уверенности
- При онбординге нового клиента — использует опыт предыдущих для генерации навыков

---

11. Event-Driven архитектура

Решение: PostgreSQL LISTEN/NOTIFY для MVP, Redis Streams для v1.0.

Почему событийная, а не линейный pipeline:

- 100 АЗС отправляют данные одновременно — нужна параллельная обработка
- При сбое одного worker-а остальные продолжают работать
- AI-агент подписывается на события ("уведоми о critical расхождении")
- Каждый шаг — отдельный worker, масштабируется независимо

```
[STS webhook]     ⟶ Event Bus ⟶ [Parser Worker]
[Bank CSV upload]  ⟶ Event Bus ⟶ [Reconciliation Worker]
[1C Feedback]      ⟶ Event Bus ⟶ [Feedback Worker]
[AI Agent]         ⟵ Event Bus ⟵ [подписка на critical]
```

Почему LISTEN/NOTIFY для MVP:

- Уже есть в PostgreSQL, ноль дополнительной инфраструктуры
- Достаточно для 200 документов/день
- Переход на Redis Streams — когда упрёмся в лимиты

---

12. Расширяемость

Добавление нового типа учёта — навыки, не код

```
1. Навык источника   → skills/sources/salary_csv.md
2. Навык сверки      → skills/reconciliation/salary_vs_timesheet.md
3. Навык учёта       → skills/accounting/salary_accrual.md
4. Навык договора    → skills/contracts/employment_contract.md (если нужно)
5. Загрузчик         → class SalaryCSVLoader(RawDataLoader) — единственный код
```

Загрузчик — единственная часть, которая требует кода (парсинг файла в JSONB). Вся бизнес-логика — в навыках. Ядро системы не меняется.

Общие справочники

Контрагенты, номенклатура, подразделения — общие для всех модулей. Одна таблица с полем `type` + JSONB metadata для специфики.

Мультиорганизация (v2.0)

Несколько юрлиц в одной базе. Поле `organization_id` в documents и accounting_documents. Каждая организация — своя учётная политика в 1С, свои справочники.

Варианты рядом с ядром (после ГСМ)

1. Сеть магазинов: ККТ + банк + эквайринг + склад + возвраты -> сверка выручки и расхождений по сменам.
2. Маркетплейсы (seller): Ozon/WB отчеты + банк + 1С + логистика -> сверка комиссий, возвратов, удержаний.
3. HoReCa/кафе-сети: POS + доставки + банк + списания по техкартам -> контроль маржи и потерь.
4. Франшизы: отчеты франчайзи в разных форматах + роялти + закупки -> единый контур сверки.
5. Транспортные компании: топливные карты + телематика + путевые листы + банк -> контроль норм расхода и сливов.
6. Стройка: акты КС/накладные + фактическое потребление + банк -> сверка материалов и перерасхода.
7. Дистрибуция: закупка + ретро-бонусы + дебиторка + банк -> автоматизация сложных условий договоров.
8. Аптеки/медрозница: касса + маркировка + поставщики + возвраты -> сверка партий и денег.
9. Сеть АЗС плюс нефуд: топливо + магазин + кафе в одной точке -> единая сверка смешанной выручки.
10. Управляющие компании ЖКХ: начисления + оплаты + банк + акты подрядчиков -> контроль кассовых разрывов и долгов.

---

13. Технологический стек

Компонент	Технология	Почему именно это
API	FastAPI + asyncpg	Async обработка 100+ станций, автодокументация OpenAPI
БД	PostgreSQL 16	ACID + JSONB + партиции — всё в одном
Валидация	Pydantic v2	Строгая типизация JSONB, быстрая, автодокументация
XML	lxml	В 5-10× быстрее стандартного ElementTree

Компонент	Технология	Почему именно это
JSON	orjson	В 5-10× быстрее стандартного json для JSONB сериализации
Очереди	PG LISTEN/NOTIFY → Redis Streams	Без лишней инфраструктуры на старте
Кэш	Redis	Сводные, сессии — когда понадобится
AI	Claude API (function calling) + **Prompt Caching**	Лучшее качество для структурированных задач учёта
AI-кэш	Anthropic Prompt Caching	Договоры, навыки, история — кэшируются. Входные токены −90%
Тесты	pytest + pytest-asyncio	Тесты сверки на реальных сценариях
Деплой	Docker Compose на VPS	spb-3 уже есть, минимум DevOps

---

14. План реализации

Scope: что в v1, а что нет

Главный принцип: **один безупречный контур «сверка → документ → 1С → feedback»** прежде, чем расширять.

В scope MVP + v1.0 (end-to-end для ГСМ):

Поток	Описание
STS API → RAW → Documents	Загрузка сменных отчётов, транзакций, замеров резервуаров
Код-сверка + AI fallback	7 типов сверки ГСМ, AI при расхождениях
Verified → XML → 1C	Custom XML, EPF-загрузчик (пресет Бухгалтерия 3.0)
1C → Feedback → ClearLedger	Подтверждение проведения или ошибка
Bank CSV → RAW → Сверка с ККТ	Эквайринг, сверка безнала
Топл. карты (TradeCorp API)	Корпоративные операции
MSTO API → Онлайн-заказы	Мобильные платежи
UI: Протокол сверки, Машина времени	Минимальный, но рабочий интерфейс

Поток	Описание
RBAC	Роли: оператор, менеджер, бухгалтер
Управленческий контур	Маржа, потери, KPI — materialized views

НЕ в scope v1 (v2.0+):

Что	Почему отложено
OCR PDF/сканов	Сроки горят. Данные для v1 — из API/CSV. PDF — как вложение-обоснование
Налоговые рекомендации AI	Юридические/репутационные риски. Сначала — стабильная бухгалтерия
Анализ договоров AI	Требует проверенного контура сверки и накопленной истории
AI-Discovery (онбординг)	Сначала — ручной онбординг первого клиента. Discovery — на основе этого опыта
Мультиорганизация	Первый клиент — одно юрлицо
Прогнозы (точка заказа)	Нужна история минимум 3 месяца
Новые отрасли (не ГСМ)	Сначала — безупречный ГСМ

MVP (4-6 недель)

Одна АЗС, сквозной pipeline RAW → сверка (код + AI) → XML → 1С → feedback. Код-обработчики делают арифметику, AI подключается для интерпретации и решений. **PDF/сканы — только как вложения, данные — из API и CSV.**

Неделя	Что делаем
1	БД: raw_data, documents, document_links. **Код:** загрузчик STS API, Pydantic-модели, статусная машина. Структура навыков (skills/)
2	**Код:** парсеры JSON→documents, математическая сверка (суммы, разницы, проценты). Первые навыки: sources/sts_shift_report.md, reconciliation/cash_vs_receipts.md. AI подключается при расхождениях
3	**Код:** расчёт НДС, построение Document Graph. Навыки учёта: accounting/fuel_retail_sale.md. Три контура из одного документа. API корректировок + audit trail
4	**Код:** генерация XML (lxml, шаблоны), файловый обмен с 1С, обработка feedback. Минимальный UI

Неделя	Что делаем
5	AI-корректировки с уверенностью. Навыки для ТТН и остатков. Тестирование на реальных данных
6	Подключение остальных АЗС. Обкатка, исправление багов

v1.0 (+4-6 недель)

Что	Детали
Код-парсеры	Bank CSV, Excel прайсы, TradeCorp API, MSTO API. Pydantic-модели для каждого типа
Навыки сверки	Все 7 типов сверки ГСМ (код считает, AI интерпретирует)
Навыки договоров	Договоры поставки: условия влияют на учёт потерь и оприходование
AI-Discovery	**Онбординг нового клиента:** AI исследует существующий учёт, генерирует черновики навыков
AI level 3	Автокорректировка типовых (>95%). Самообучение на истории корректировок
UI	Дашборд здоровья. Протокол сверки. Три контура учёта рядом

v2.0 (+6-8 недель)

Что	Детали
Онбординг-конвейер	Полный цикл: Discovery → генерация навыков → валидация → параллельная работа → переключение
Налоговые рекомендации	AI подсвечивает варианты отражения и риски. Решение — за специалистом
OCR документов	PDF ТТН, сканы накладных. Отдельный этап после стабильной работы сверок
Мультиорг	Несколько юрлиц. Консолидация. Разные учётные политики
Новые отрасли	За пределы ГСМ: торговля, услуги, производство — навыки, не код
AI	Предлагает новые навыки на основе паттернов. Прогнозы. Библиотека навыков растёт с каждым клиентом
Масштаб	Event-driven (Redis Streams). Партиционирование

15. Риски

Риск	Вероятность	Что делаем
JSONB превращается в хаос без дисциплины	Высокая	Pydantic-модели на каждый doc_type + `metadata_version` в каждой записи + generated columns для accounting-critical полей (total_amount, vat_amount, doc_date, counterparty_inn) с CHECK constraints на уровне БД. Миграция старых версий — в нормализаторе. См. LAYER2_ARCHITECTURE.md
1С-интеграция ломается при обновлении	Средняя	Версионирование XML, тесты на sandbox-базе 1С
AI делает неправильные корректировки	Средняя	Порог уверенности 95%, полный audit trail, ролбэк
OCR ТТН — низкое качество	Высокая	Fallback на ручной ввод, постепенное обучение модели
Справочники ClearLedger и 1С рассинхронизируются	Средняя	Автосинхронизация перед каждой выгрузкой
Онбординг затягивается: бизнес сопротивляется	Высокая	Параллельная работа, не заменяем сразу, показываем ценность
AI-Discovery неточно понимает процессы клиента	Средняя	Фаза валидации человеком обязательна, навыки — черновики до утверждения
AI галлюцинирует на грязных данных (битые CSV, CP1251, сканы)	Высокая	Визуальный редактор навыков, прогон на реальных данных, основание в каждом навыке
Нетиповая 1С клиента не принимает наш XML	Высокая	EPF-загрузчик с настраиваемым маппингом, анализ структуры 1С при онбординге
Донастройка нетиповой 1С выходит за сроки и ломает rollout	Высокая	Sandbox-прогон до production, протокол совместимости XML/EPF, SLA с 1С-специалистом
Корректировка после закрытия периода в 1С	Средняя	Автопроверка даты запрета, корректирующие документы в текущем периоде

Риск	Вероятность	Что делаем
JSONB раздувает базу при частых обновлениях	Средняя	Агрессивный autovacuum, статус в отдельной колонке, TOAST, мониторинг dead_tup
TOAST/архив не контролируются, БД дорожает и деградирует	Средняя	Мониторинг TOAST, ретенция партиций 12 месяцев, архив в S3-совместимое хранилище
AI Discovery «поймёт структуру 1С» — утопия	Высокая	Стандартные пресеты EPF для Бухгалтерии 3.0 (80% клиентов), ручная настройка 1С-ником
Конфликт версий EPF: ClearLedger обновился, EPF нет	Средняя	Версия XML-схемы + проверка совместимости в EPF + graceful error → папка error/
Некачественный вход (битые CSV/кодировки/OCR) приводит к ложным решениям	Высокая	Input quality score, manual_validation queue, стоп автопроведения при score <70
Деградация навыка после обновления незаметно уходит в production	Средняя	Role-based review, canary 10→50→100, авт rollback по метрикам
Юридические риски по налоговым рекомендациям AI	Средняя	Шаблоны согласования, явный дисклеймер ответственности, обязательное утверждение специалистом
Саботаж внедрения менеджментом (прозрачность невыгодна)	Высокая	Продавать собственнику. Параллельная работа без принудительного переключения
Context window cost: договор 50 стр. + история = дорого	Средняя	Anthropic Prompt Caching: кэш договоров и навыков, входные токены −90%

---

16. Почему AI — с первого дня, а не "потом"

Классический аргумент: "Сначала сделаем систему, потом добавим AI."
Почему это ошибка: AI в 2026 — не премиум-фича. Это базовый инструмент дешевле любого ручного труда.

```
Стоимость компонентов ClearLedger в месяц (гибридный подход):

  VPS (2 CPU, 4GB RAM)                    ~1 500₽
  PostgreSQL                                    0₽  (Docker)
  Код-обработчики (95% операций)                0₽  (Python на том же VPS)
  AI-сверка расхождений (Haiku, ~2K запросов)  ~75₽  ($0.84)
  AI-аналитика (Sonnet, 500 запросов)       ~1 100₽  ($12)
  AI-онбординг нового клиента (разовый)     ~2 000₽  ($22)
  ──────────────────────────────────────────────────
  Итого ежемесячно:     ~2 675₽/месяц
  (при онбординге: +2 000₽ разово)

Стоимость ручной сверки:
  Менеджер 1-3 ч/день × 22 дня  = 10 000-30 000₽/месяц
  Ошибки, штрафы, пересдача      = непредсказуемо
```

Код делает 95% работы мгновенно и бесплатно. AI подключается для 5% нестандартных случаев и стоит ~75₽/месяц вместо ~220₽. Менеджер занимается исключениями и решениями, а не рутиной.

Нюанс: context window cost. Простая сверка = 1600 токенов. Но если AI читает договор на 50 страниц + историю за месяц для анализа паттерна — запрос вырастает до 50-100K токенов.

Решение: Anthropic Prompt Caching. Договоры, навыки и исторические паттерны кэшируются. Входные токены из кэша стоят на **90% дешевле**. Договор читается один раз и остаётся в кэше 5 минут — все последующие сверки по этому контрагенту используют кэш.

```
Без кэширования:
  Договор (50K токенов) × 20 сверок/день = 1M входных токенов/день
  Haiku: ~$0.25/день → ~$7.5/месяц

С кэшированием:
  Договор кэшируется: 50K × $0.03/M = $0.0015 (один раз)
  19 повторных: 50K × $0.003/M × 19 = $0.003/день
  Итого: ~$0.15/день → ~$4.5/месяц (экономия 40%)
  На Sonnet экономия существеннее: входные токены -90%
```

Шесть свойств архитектуры, которые делают AI эффективным:

1. **Данные доступны через API** — AI работает с данными напрямую, не через UI

2. **JSONB = самодокументируемость** — AI читает структуру из самих данных, не нужно описывать схему из 50 таблиц
3. **Навыки (Skills) вместо кода** — бизнес-логика в текстовых описаниях, AI может предлагать новые навыки и корректировать существующие
4. **Document Graph** — AI находит причины расхождений по цепочке связей между документами
5. **Иммутабельная история** — AI учится: какие расхождения ложные, какие корректировки типовые, какие паттерны сезонные
6. **Prompt Caching** — договоры, навыки, схемы кэшируются, повторные запросы на 90% дешевле

Расхождения не зависят от масштаба. Даже на одной АЗС за смену: касса может не сойтись, банковский реестр задержался, ТТН не совпала с фактом. Это природа бизнеса с наличными и физическим товаром. ClearLedger нужен не "когда станций станет много", а с момента, когда есть хотя бы одна станция и хотя бы один источник расхождений.